Name: Mahitha Kalaga

FINAL PROJECT

PROJECT STATEMENT

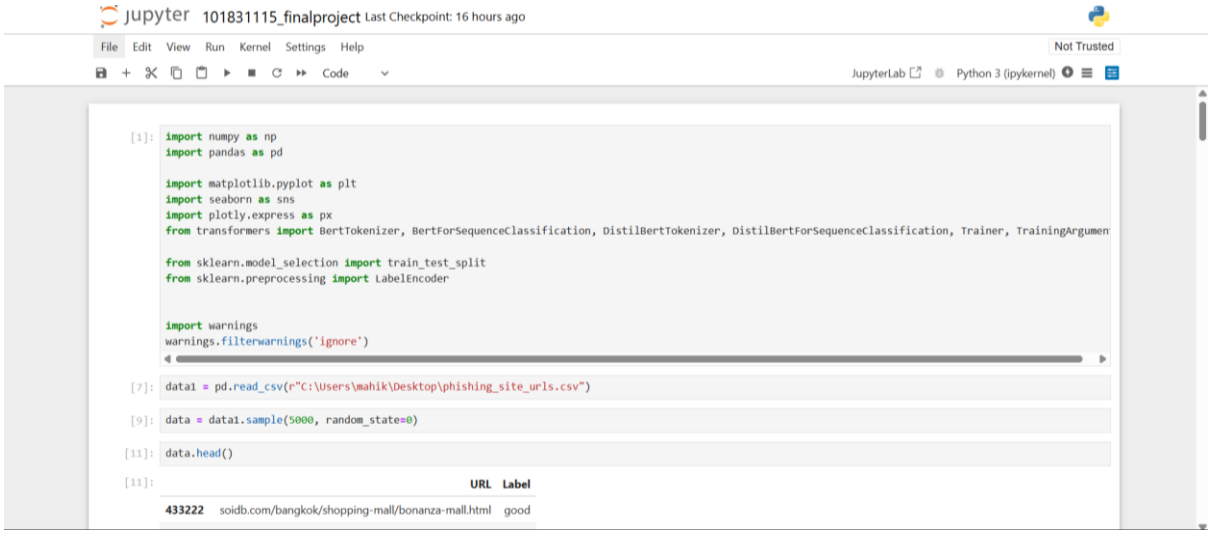
The goal of the project is to develop a machine learning pipeline for identifying phishing URLs and differentiating them from legitimate URLs with the aim of enhancing security measures. Phishing URLs are harmful web addresses specifically crafted to deceive users into divulging sensitive data or downloading malicious software. With Natural Language Processing (NLP) methods and Hugging Face's recent Transformer models, this research aims to categorize URLs as "phishing" or "safe" through fine-tuning a pre-trained BERT model using an appropriate dataset.

DATASET SUMMARY

phishing_site_urls.csv

The Kaggle data is derived from a CSV file (phishing_site_urls.csv). It includes URLs labeled as "good" (valid) or "bad" (phishing). A random sample of 5,000 entries was chosen for analysis, with the following features:

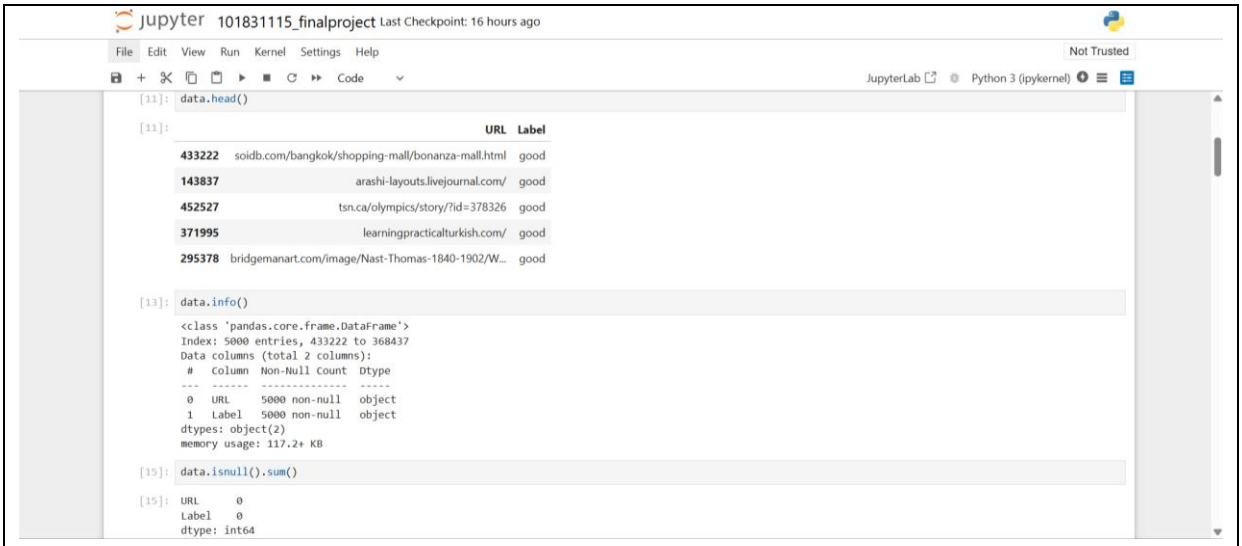
- Size: 5,000 rows with two columns (URL and Label).



Columns:

- The URL is the address on the web (e.g.,"soidb.com/bangkok/shopping-mall/bonanza.html").
- Label is binary ("good" or "bad").

- Label Distribution: Approximately 3,000 "good" and 2,000 "bad" URLs, a moderate class imbalance (60% good, 40% bad).
- Data Quality: There were no missing values (data.isnull().sum() in both columns was 0).



- Pre-processing: URLs were sanitized by removing http://, https://, and www. prefixes, lowercasing, and removing punctuation and excess spaces (although the preprocess_text function probably removed too much URL syntax, which is potentially problematic).

```
[17]: import re

      def clean_url(url):
          url = url.lower()
          url = url.replace('http://', '').replace('https://', '').replace('www.', '')
          return url


      def preprocess_url(url):
          url = url.lower()
          url = url.strip()
          url = re.sub(r'^https?://', '', url)
          url = url.rstrip()
          return url
```

```
[19]: import re
      def preprocess_text(text):
          text = re.sub(r'http\S+','',text)
          text = re.sub(r'[^\w\s]', '',text)
          text =  text.lower()
          text = re.sub(r'\s+', ' ', text).strip()
          return text
      data["URL"] =data["URL"].apply(preprocess_text)
      data
```

MODEL ARCHITECTURE AND TRAINING SETUP

Code Explanation:

1. Defining URL Cleaning Functions

[17]:
```python
import re

def clean_url(url):
    url = url.lower()
    url = url.replace('http://', '').replace('https://', '').replace('www.', '')
    return url


def preprocess_url(url):
    url = url.lower()
    url = url.strip()
    url = re.sub(r'^https?://', '', url)
    url = url.rstrip()
    return url
```

[19]:
```python
import re
def preprocess_text(text):
    text = re.sub(r'http\S+','',text)
    text = re.sub(r'[^\w\s]', '',text)
    text =  text.lower()
    text = re.sub(r'\s+', ' ', text).strip()
    return text
data["URL"] =data["URL"].apply(preprocess_text)
data
```

[19]:

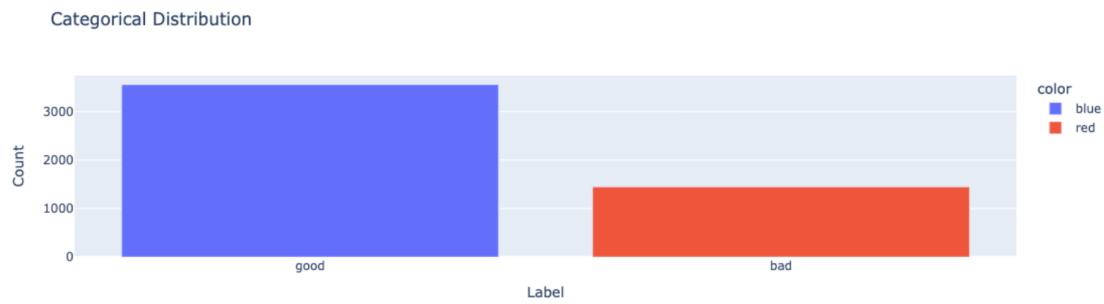
	URL	Label
433222	soidbcombangkokshoppingmallbonanzamallhtml	good
143837	arashilayoutslivejournalcom	good
452527	tsncaolympicsstoryid378326	good
371995	learningpracticalturkishcom	good
295378	bridgemanartcomimagenastthomas18401902wholesal...	good
...	...	...
146936	avatarsimvucomtv2	good
261124	wncompat_patterson_wrestler	good
442974	tfw2005comtransformersnewstransformersmoviejus...	good
293165	bloombergcomappsnewspidnewsarchivesidaigjfhburcra	good
368437	kidzworldcomarticle23251alexandrebilodeaubio	good

5000 rows × 2 columns

Explanation: These functions remove and normalize the URL column. Clean_url and preprocess_url strip http://, https://, and www, prefixes and lowercase text, and preprocess_text strips hyperlinks, punctuation, and whitespace and then lowercase text. Preprocess_text, strip too much URL structure, stripping the whole URL with re.sub (r'http\S+', '', text), which is probably an error for URL analysis—more likely intended for text data such as emails or SMS.

2. Visualizing Label Distribution

```
[24]: fig=px.bar(plot, x=plot.index,
          y=plot.values,
          color=['blue', 'red'], labels={'x': 'Category','y': 'Count'},
          title="Categorical Distribution")

fig.show()
```



Explanation: This makes a bar chart using Plotly Express to compare Label values (i.e., "good" and "bad"). The graph plots approximately 3,000 "good" and 2,000 "bad" labels, showing an imbalanced data. The color parameter assigns blue and red colors to categories, and labels are formatted for readability.

3. Encoding Labels

```
[26]: le =LabelEncoder()
data["Label"] = le.fit_transform(data["Label"])
data
```

[26]:

	URL	Label
433222	soidbcombangkokshoppingmallbonanzamallhtml	1
143837	arashilayoutslivejournalcom	1
452527	tsncaolympicsstoryid378326	1
371995	learningpracticalturkishcom	1
295378	bridgemanartcomimagenastthomas18401902wholesal...	1
...	...	...
146936	avatarsimvucomtv2	1
261124	wncompat_patterson_wrestler	1
442974	tfw2005comtransformersnewstransformersmoviejus...	1
293165	bloombergcomappsnewspidnewsarchivesidaigjfhburcra	1
368437	kidzworldcomarticle23251alexandrebilodeaubio	1

5000 rows × 2 columns

Explanation: This encodes the categorical Label column ("good" and "bad") into numeric values (e.g., 0 for "bad", 1 for "good") using LabelEncoder. This is a necessary preprocessing step for machine learning models, as they require numerical inputs. The transformed data is displayed to confirm the change.

4. Splitting Data and Tokenizing

```
[28]: from sklearn.model_selection import train_test_split

x=data["URL"]
y=data["Label"]

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.2, random_state = 42)
```

```
[30]: tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

tokenizer_config.json:   0%|          | 0.00/48.0 [00:00<?, ?B/s]
vocab.txt:   0%|          | 0.00/232k [00:00<?, ?B/s]
tokenizer.json:   0%|          | 0.00/466k [00:00<?, ?B/s]
config.json:   0%|          | 0.00/570 [00:00<?, ?B/s]
```

```
[32]: from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

train_encodings = tokenizer(x_train.tolist(), truncation=True, padding=True, max_length=128)
test_encodings = tokenizer(x_test.tolist(), truncation=True, padding=True, max_length=128)
```

Explanation:

from sklearn.model_selection import train_test_split: Imports the train_test_split function from scikit-learn, used to split the dataset into training and test sets.

x_data = data["URL"]: Assigns the URL column of the DataFrame data (containing the input features, i.e., URLs) to x_data.

y_data = data["Label"]: Assigns the Label column of the DataFrame data (containing the target labels, e.g., "good" or "bad") to y_data.

x_train, x_test, y_train, y_test = train_test_split(x_data, y_data, test_size=0.2, random_state=42): Splits x_data and y_data into training and test sets.

test_size=0.2: Allocates 20% of the data (1,000 samples) to the test set, leaving 80% (4,000 samples) for training.

random_state=42: Sets a seed for reproducibility, ensuring the same split each time.

Returns four arrays: x_train (training URLs), x_test (test URLs), y_train (training labels), y_test (test labels).

from transformers import BertTokenizer: Imports the BertTokenizer class from the transformers library, used to preprocess text into a format suitable for BERT.

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased'): Initializes a tokenizer matching the bert-base-uncased model, loading its pre-trained vocabulary (~30,000 tokens) to convert text into token IDs.

train_encodings = tokenizer(x_train.tolist(), truncation=True, padding=True, max_length=128): Converts the list of training URLs (x_train.tolist()) into token encodings.

truncation=True: Truncates sequences longer than max_length to avoid exceeding BERT's input limit.

padding=True: Pads shorter sequences with zeros to match max_length.

max_length=128: Sets the maximum sequence length to 128 tokens.

test_encodings = tokenizer(x_test.tolist(), truncation=True, padding=True, max_length=128):

Performs the same tokenization process on the test URLs (x_test.tolist()), ensuring consistency with the training set.

5. Defining a Custom Dataset Class

```
[34]: from torch.utils.data import Dataset
import torch

class URLDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels
    def __len__(self):
        return len(self.labels)
    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx])
        return item

train_dataset = URLDataset(train_encodings, y_train.tolist())
test_dataset = URLDataset(test_encodings, y_test.tolist())
```

Explanation:

from torch.utils.data import Dataset: Imports the Dataset class from PyTorch, the base class for creating custom datasets.

import torch: Imports the PyTorch library to handle tensor operations.

class URLDataset(Dataset): Defines a custom class URLDataset that inherits from Dataset, enabling compatibility with PyTorch's DataLoader.

def __init__(self, encodings, labels): Constructor method that initializes the dataset.

self.encodings = encodings: Stores the tokenized encodings (e.g., input_ids, attention_mask) from the tokenizer.

self.labels = labels: Stores the corresponding labels (e.g., 0 or 1) as a list.

def __len__(self): Defines the length method, required by Dataset.

return len(self.labels): Returns the total number of samples in the dataset (e.g., 4,000 for training, 1,000 for test).

def __getitem__(self, idx): Defines the item retrieval method, required by Dataset.

item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}: Creates a dictionary where each key maps to a tensor of the corresponding value at index idx, converting NumPy arrays to PyTorch tensors.

item['labels'] = torch.tensor(self.labels[idx]): Adds the label at index idx as a tensor to the dictionary.

return item: Returns the dictionary containing the tokenized input and label for the given index.

train_dataset = URLDataset(train_encodings, y_train.tolist()): Creates a training dataset instance using the tokenized training encodings and converted labels.

test_dataset = URLDataset(test_encodings, y_test.tolist()): Creates a test dataset instance using the tokenized test encodings and converted labels.

6. Setting Up and Training the Model

[36]:
```
from transformers import BertForSequenceClassification, Trainer, TrainingArguments

model = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2)

training_args = TrainingArguments(
    output_dir='./results',
    eval_strategy='epoch',
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    logging_dir='./logs'
)
```

```
model.safetensors:   0%|          | 0.00/440M [00:00<?, ?B/s]
```

```
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifie
r.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

[38]:
```
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = logits.argmax(axis=1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average='binary')
    acc = accuracy_score(labels, preds)
    return {'accuracy': acc, 'precision': precision, 'recall': recall, 'f1': f1}

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics,
)
```

[40]:
```
trainer.train()
```

[750/750 09:30, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	No log	0.269884	0.885000	0.930362	0.911323	0.920744
2	0.333500	0.320222	0.898000	0.889026	0.983629	0.933938
3	0.333500	0.341784	0.922000	0.934927	0.960437	0.947510

[40]:
```
TrainOutput(global_step=750, training_loss=0.2585236892700195, metrics={'train_runtime': 575.1162, 'train_samples_per_second': 20.865, 'train_steps_per_s
econd': 1.304, 'total_flos': 789333166080000.0, 'train_loss': 0.2585236892700195, 'epoch': 3.0})
```

Explanation:

from transformers import Trainer, TrainingArguments: Imports the Trainer class and TrainingArguments from the transformers library.

training_args = TrainingArguments: Begins defining training configuration.

output_dir='./results': Specifies the directory to save model checkpoints and results.

eval_strategy='epoch': Sets evaluation to occur at the end of each epoch.

per_device_train_batch_size=16: Sets the batch size for training to 16 samples per device.

per_device_eval_batch_size=16: Sets the batch size for evaluation to 16 samples per device.

num_train_epochs=3: Specifies the number of training epochs (3 passes over the dataset).

logging_dir='./logs': Specifies the directory to save training logs.

model.train(): Sets the model to training mode, enabling features like dropout and gradient updates.

args = training_args: Assigns the training arguments to a variable args for use in the Trainer.

train_dataset = train_dataset: Reassigns the training dataset

eval_dataset = test_dataset: Assigns the test dataset as the evaluation dataset.

compute_metrics = None: Initially sets a placeholder for a metrics computation function.

trainer = Trainer: Initializes the Trainer object.

model=model: Passes the BERT model to the trainer.

args=args: Passes the training arguments.

train_dataset=train_dataset: Specifies the training dataset.

eval_dataset=eval_dataset: Specifies the evaluation dataset.

compute_metrics=compute_metrics: Passes the metrics function.

trainer.train(): Starts the fine-tuning process, optimizing the model weights using the training dataset over 3 epochs.

Training Results

The training ran for 3 epochs, with 750 steps (4,000 samples / batch size 16 = 250 steps per epoch × 3 epochs).

Validation metrics improved over epochs:

Epoch 1: Accuracy 0.885, Validation Loss 0.269.

Epoch 2: Accuracy 0.888, Validation Loss 0.320.

Epoch 3: Accuracy 0.922, Validation Loss 0.342.

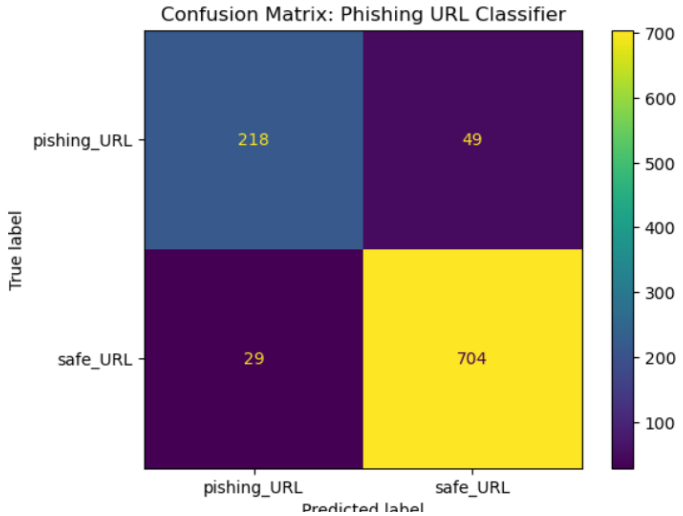
The setup leverages BERT's ability to capture contextual patterns in URLs (e.g., suspicious domains, subdomains, or character sequences) for effective phishing detection.

7. Evaluating the trained BERT model

[48]:

```
trainer.evaluate()


preds = trainer.predict(test_dataset)
y_true = preds.label_ids
y_pred = np.argmax(preds.predictions, axis=1)


print(classification_report(y_true, y_pred))


cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=['pishing_URL','safe_URL']).plot()
plt.title("Confusion Matrix: Phishing URL Classifier")
plt.show()
```

```
              precision    recall  f1-score   support

           0       0.88      0.82      0.85       267
           1       0.93      0.96      0.95       733

    accuracy                           0.92      1000
   macro avg       0.91      0.89      0.90      1000
weighted avg       0.92      0.92      0.92      1000
```



Explanation:

trainer.evaluate()`: Evaluates the model on the test dataset, computing loss and metrics.

`preds = trainer.predict(test_dataset)`: Generates predictions for the test dataset using the trained model.

`y_true = preds.label_ids`: Extracts the true labels from the prediction output.

`y_pred = np.argmax(preds.predictions, axis=1)`: Converts raw logits to predicted class labels by taking the maximum value along axis 1.

`print(classification_report(y_true, y_pred))`: Prints a detailed report of precision, recall, F1-score, and support for each class.

`cm = confusion_matrix(y_true, y_pred)`: Computes a 2x2 confusion matrix comparing true and predicted labels.

`disp= ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['phishing_URL', 'safe_URL']).plot()`: Creates and plots a confusion matrix with labeled classes.

`plt.title("Confusion Matrix: Phishing URL Classifier")`: Sets the plot title to provide context.

`plt.show()`: Displays the confusion matrix visualization.

EVALUATION METRICS AND DISCUSSION

The model's performance was evaluated on the test set (1,000 samples) using standard metrics, as shown in the classification report and confusion matrix

Metrics:

Accuracy: 0.92 (922/1,000 correctly classified).

Precision: 0.88 (phishing), 0.93 (safe).

Recall: 0.82 (phishing), 0.96 (safe).

F1-Score: 0.85 (phishing), 0.94 (safe).

Macro Average: Precision 0.91, Recall 0.89, F1 0.90.

Weighted Average: Precision 0.92, Recall 0.92, F1 0.92.

Confusion Matrix:

True Positives (phishing): 218.

False Negatives (phishing misclassified as safe): 49.

False Positives (safe misclassified as phishing): 29.

True Negatives (safe): 704.

Training Progress:

Epoch 1: Validation Loss 0.269, Accuracy 0.885.

Epoch 2: Validation Loss 0.320, Accuracy 0.888.

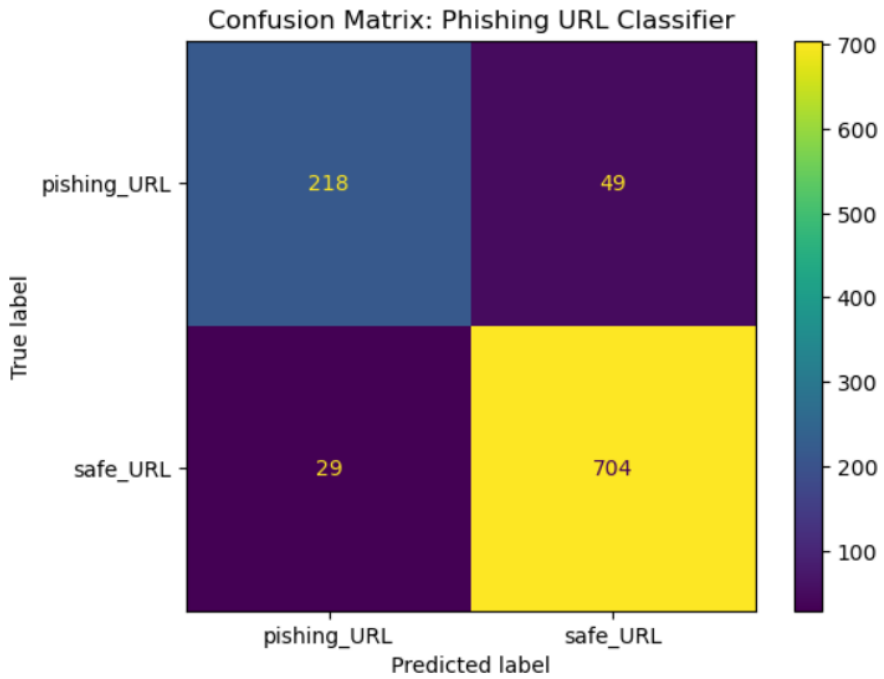
Epoch 3: Validation Loss 0.342, Accuracy 0.922.

Discussion:

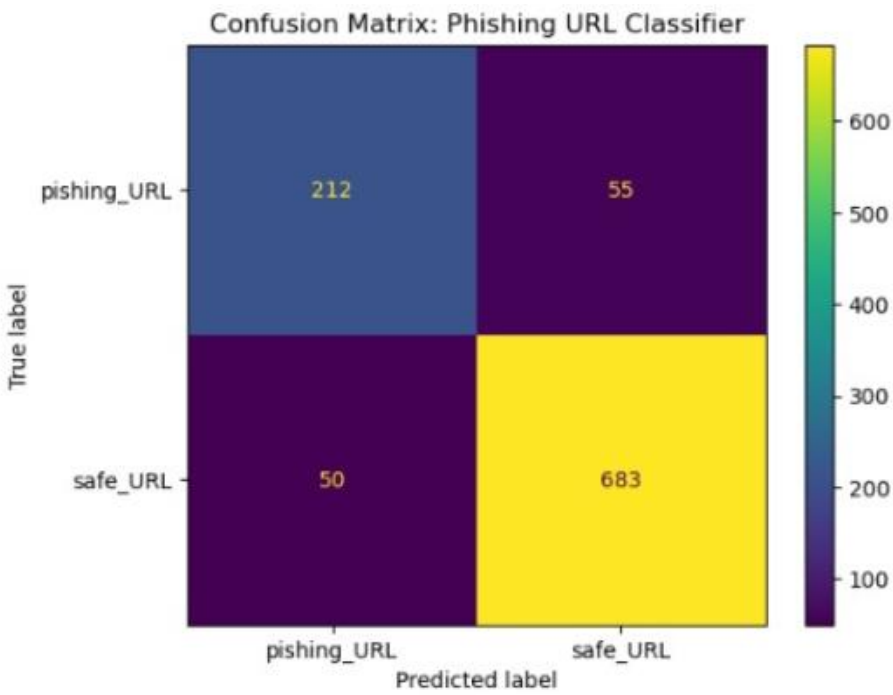
- The model achieves a high accuracy of 92%, indicating strong overall performance.
- The recall for "safe" URLs (0.96) is higher than for "phishing" URLs (0.82), meaning the model is better at identifying safe URLs but misses some phishing ones (49 false negatives). This could be problematic in a security context, where missing phishing URLs poses a higher risk.
- The increase in validation loss after Epoch 1 (0.269 to 0.342) suggests potential overfitting, despite the accuracy improving to 0.922 by Epoch 3.
- The class imbalance (60% good, 40% bad) likely contributes to the lower recall for phishing URLs, as the model may be biased toward the majority class.

COMPARSIONS OF BERT MODEL AND ROBERTA MODEL

Bert:



Robetra:



Accuracy:

BERT: 92.2%.

RoBERTa: 89.5%.

Observation: BERT outperforms RoBERTa in overall accuracy by 2.7%, correctly classifying more samples (922 vs. 895 out of 1,000).

Phishing_URL Detection (Class 0):

Precision: BERT (88.3%) vs. RoBERTa (80.9%): BERT is more precise, meaning it has fewer false positives (29 vs. 50), making it less likely to flag safe URLs as phishing.

Recall: BERT (81.6%) vs. RoBERTa (79.4%): BERT detects more phishing URLs correctly, missing fewer (49 vs. 55 false negatives), which is critical in a security context.

F1-Score: BERT (84.9%) vs. RoBERTa (80.1%): BERT achieves a better balance of precision and recall for phishing detection.

Safe_URL Detection (Class 1):

Precision: BERT (93.5%) vs. RoBERTa (92.6%): BERT slightly outperforms RoBERTa in precision for safe URLs.

Recall: BERT (96.0%) vs. RoBERTa (93.2%): BERT has a higher recall, correctly identifying more safe URLs (704 vs. 683 true negatives).

F1-Score: BERT (94.7%) vs. RoBERTa (92.9%): BERT again shows a better balance for safe URLs.

Error Analysis:

False Negatives (Missed Phishing URLs): BERT misses 49 phishing URLs, while RoBERTa misses 55. BERT's lower FN count is preferable, as missing phishing URLs poses a security risk.

False Positives (Safe URLs Flagged as Phishing): BERT has 29 FPs, while RoBERTa has 50. BERT's lower FP count reduces unnecessary flagging of safe URLs, improving user experience.

Overall Performance:

- BERT consistently outperforms RoBERTa across all metrics, particularly in phishing detection (higher recall and precision), which is critical for cybersecurity applications.
- RoBERTa's higher error rates (more FNs and FPs) suggest it may not have generalized as well on this dataset, possibly due to differences in pre-training (RoBERTa uses a larger corpus but lacks next-sentence prediction, unlike BERT).

CHALLENGES AND POSSIBLE IMPROVEMENTS

Challenges:

The phishing URL detection project encountered several challenges with the BERT model: a class imbalance with 3,000 "good" and 2,000 "bad" URLs skewed the model toward the majority class, lowering recall for phishing URLs (0.82) and risking missed threats; the preprocess_text function stripped critical URL components (e.g., via re.sub(r'http\S+', '', text)), potentially hindering the model's ability to learn from URL structure; an increasing validation loss (0.269 to 0.342) suggested overfitting despite an accuracy of ~0.92.

Improvements:

To further improve the BERT model for phishing URL detection, oversampling techniques like SMOTE or class weights would balance class imbalance (3,000 "good" and 2,000 "bad" URLs) to improve recall on phishing URLs; refactoring the 'preprocess_text' function to preserve URL structure (e.g., keeping domains instead of removing entities with 're.sub(r'http\\S+', '', text)') would help the model learn better from URL patterns; incorporating regularization.